Window Functions, Explained Simply

A plain-language version of Lecture 3

Gregory S. DeLozier, PhD

One Small Table

title	year	rating
A	2000	8.0
B	2000	8.0
C	2000	7.0
D	2002	9.0
E	2002	6.0

Five movies. We will use only this table.

GROUP BY Folds Rows Together

```
SELECT year, avg(rating)
FROM movies
GROUP BY year;
```

year	avg_rating
2000	7.7
2002	7.5

Five rows in. Two rows out. The movies are gone.

What If We Want Both?

title	year	rating	year_avg
A	2000	8.0	7.67
B	2000	8.0	7.67
C	2000	7.0	7.67
D	2002	9.0	7.50
E	2002	6.0	7.50

Every movie, with its year's average beside it.

The Whole Idea

A window function adds a column.

Each row computes that column by looking at a group of related rows: its window.

No rows are removed.

Reading OVER

```
function() OVER (
    PARTITION BY ...
    ORDER BY ...
    ROWS BETWEEN ... AND ...
)
```

Part	Question it answers
PARTITION BY	Which rows are in my group?
ORDER BY	In what order do my group's rows line up?
ROWS BETWEEN	How much of the line can I see from my spot?

All three are optional.

OVER () Means Everyone

```
SELECT title, year, rating,
       avg(rating) OVER () AS avg_all
FROM movies;
```

title	year	rating	avg_all
A	2000	8.0	7.6
B	2000	8.0	7.6
C	2000	7.0	7.6
D	2002	9.0	7.6
E	2002	6.0	7.6

Empty OVER: every row sees all five rows.

PARTITION BY Makes Smaller Groups

```
SELECT title, year, rating,
       avg(rating) OVER (PARTITION BY year) AS year_avg
FROM movies;
```

title	year	rating	year_avg
A	2000	8.0	7.67
B	2000	8.0	7.67
C	2000	7.0	7.67
D	2002	9.0	7.50
E	2002	6.0	7.50

Each row sees only the rows from its own year.

GROUP BY Versus a Window

	GROUP BY	Window function
Rows out	One per group	One per input row
Individual rows	Gone	Kept
Typical use	Summarize	Compare each row with others

Choose by the question: summarize, or compare?

Ranking: Handing Out Places

```
SELECT title, year, rating,
  dense_rank() OVER (
    PARTITION BY year
    ORDER BY rating DESC
  ) AS place
FROM movies;
```

Partition: a separate race for each year. ORDER BY: best rating first.

Three Ways to Rank a Tie

title	rating	dense_rank	rank	row_number
A	8.0	1	1	1
B	8.0	1	1	2
C	7.0	2	3	3

dense_rank: no gaps. rank: skips places. row_number: never ties.

Why WHERE Cannot Use a Rank

Order of work	Step
1	FROM and JOIN
2	WHERE
3	GROUP BY
4	Window functions
5	SELECT list and ORDER BY

WHERE runs before the rank exists.

Filter After Ranking

```
WITH ranked AS (
  SELECT title, year, rating,
    dense_rank() OVER (
      PARTITION BY year
      ORDER BY rating DESC) AS place
  FROM movies
)
SELECT * FROM ranked WHERE place = 1;
```

title	year	rating	place
A	2000	8.0	1
B	2000	8.0	1
D	2002	9.0	1

The inner query makes the rank. The outer query uses it.

lag: Look at the Row Above

```
lag(movies) OVER (ORDER BY year)
```

year	movies	lag(movies)
2000	3	none
2002	2	3

Line the rows up. lag reads the one just above. The first row has nothing above it.

Windows Over Grouped Rows

```
WITH yearly AS (
  SELECT year, count(*) AS movies
  FROM movies GROUP BY year
)
SELECT year, movies,
  movies - lag(movies) OVER (ORDER BY year) AS change
FROM yearly;
```

year	movies	change
2000	3	none
2002	2	-1

Group first: one row per year. Then compare those rows.

Running Total: The Frame

```
sum(movies) OVER (
  ORDER BY year
  ROWS BETWEEN UNBOUNDED PRECEDING
           AND CURRENT ROW
)
```

year	movies	rows included	running_total
2000	3	2000	3
2001	1	2000, 2001	4
2002	2	2000, 2001, 2002	6

Frame: from the first row down to me.

Cheat Sheet

Question	Tool
Who is first, second, third in each group?	dense_rank, rank, row_number
What was the value in the row above?	lag
What is the total so far?	sum with a frame
How does this row compare with its group?	avg with PARTITION BY

Pick the tool from the question.

Three Common Mistakes

No PARTITION BY: one big group.

ORDER BY inside OVER: it sets the line-up for the calculation, not the order of the output.

Rank in WHERE: use an inner query, then filter outside.

Try It

```
psql "$DATABASE_URL" -f lecture-03-windows.sql
```

Change `dense_rank` to `rank`. What changes?

Remove `PARTITION BY`. What changes?

Predict first. Then run it.